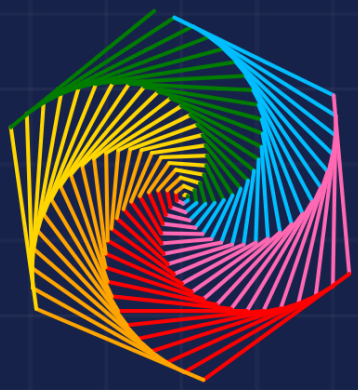
THE KIDAI EASY GUIDE · AGES 8 AND UP

# Code with Sparky

Learn to talk to computers in plain English, draw pictures with a robot, and teach your very own AI.





**KidAI Easy 1.2** · Developed by Sourabh Somani

Runs on your device. No internet needed.

This book belongs to:

_____

# Hi Hello, I'm Sparky!

Let's learn how this book works.

I'm a little robot, and I do **exactly** what you tell me. The instructions you give me are called **code**. When you write code, you are a **programmer**!

We will talk in a language called **KidAI Easy**. It looks a lot like English, so you can read it out loud. Here is a whole program:

CODE

```
say "Hello! I am Sparky."
say "Let's be friends!"
```

OUTPUT

```
Hello! I am Sparky.
Let's be friends!
```

## How to run your code

### 1. Type

Open the **KidAI Easy Playground**. Type your code in the big box on the left.

### 2. Run

Press the green **Run** button. On a keyboard you can press Ctrl + Enter too.

### 3. Watch

See what I say under **Sparky says**, and what I draw on **Sparky's stage**.

Pressing the red **Stop** button always stops me. In this book, every code box has an **output** box next to it. It shows what really happens when you run it.

**Sparky:** The playground is at **sourabhsomani.github.io/kidai-easy**. On a computer you can also write code in **VS Code** and run it with the `kidai` command. If your program draws, plays music or is a game, a stage window opens so you can still see and hear me!

## Four golden rules

### 1

**One instruction per line**

Each line tells me one thing to do. I go from the top line to the bottom line.

### 2

**Words go in "quote marks"**

`say "cat"` shows the word cat. Without quotes I think `cat` is something I should remember.

3

### Blocks start with : and finish with end

A line ending in `:` starts a group of instructions. The word `end` closes it. Push the lines inside two spaces to the right so they look neat.

4

### # is a note for humans

Anything after `#` is a note. I skip it. Use notes to explain your code.

## What's inside


1 Make Sparky talk 4
2 Remember things 5
3 Ask questions 7
4 Make choices 8
5 Repeat with loops 10
6 Lists and random surprises 12
7 Draw with Sparky 14
8 Make your own recipes 16
9 Buttons and timing 18
10 Make a game 20
11 Teach an AI 24
12 Is the AI right? 26
13 Pictures, numbers and big AI 29
14 Records, words and saving 32
15 Watch your code run 34
16 Fix bugs like a pro 37
17 Share and take on challenges 39
18 Big projects 40
A–Z Sparky's word book 43
★ Your coder certificate 45
★ Meet the developer 46
i For grown-ups 47

# 1 Make Sparky talk

Your first instruction: say

The word `say` makes me show something. Put words inside quote marks:

CODE

```
say "Hi there!"
say "I like pizza."
```

OUTPUT

```
Hi there!
I like pizza.
```

I am also a super fast calculator. Numbers don't need quote marks:

CODE

```
say 2 + 3
say 10 * 4
say "2 + 3"
```

OUTPUT

```
5
40
2 + 3
```

**Sparky:** Look at the last line! `"2 + 3"` is in quote marks, so I show the words. I don't do the maths.

## Joining things together

Use `+` to stick words and numbers together into one sentence. Remember to put spaces inside the quote marks:

CODE

```
say "I am " + 9 + " years old."
say "Ha" * 3
```

OUTPUT

```
I am 9 years old.
HaHaHa
```

A word times a number repeats it. Ha ha ha!

**Try it!**

- [ ] Make Sparky say your name and your favourite food.
- [ ] Make Sparky work out 365 × 12.
- [ ] Make Sparky laugh 10 times using `*`.

# 2 Remember things

Variables are boxes with names

A **variable** is like a box with a name written on it. You can put a number or some words inside, and look inside later.

`set` makes a box and puts something in it. `change ... by` adds to a number.

CODE

```
set score to 0
say "Score: " + score
change score by 10
change score by 5
say "Score: " + score
```

OUTPUT

```
Score: 0
Score: 15
```

Use `set` again to put something new in the box. The old thing goes away:

CODE

```
set pet to "cat"
say "My pet is a " + pet
set pet to "dragon"
say "Now my pet is a " + pet
```

OUTPUT

```
My pet is a cat
Now my pet is a dragon
```

**Sparky:** Good names tell you what is inside: `score`, `lives`, `player_name`. Names can't have spaces, so use an underscore _ instead.

## Maths signs

| Sign | What it does | Example |
|---|---|---|
| `+` | add | `5 + 2` is 7 |
| `-` | take away | `5 - 2` is 3 |
| `*` | times | `5 * 2` is 10 |
| `/` | divide | `10 / 4` is 2.5 |
| `mod` | the remainder after dividing | `10 mod 3` is 1 |
| `( )` | do this part first | `(2 + 3) * 4` is 20 |

CODE

```
set sweets to 23
set friends to 4
say "Each friend gets " + round(sweets / friends)
say "Left over: " + sweets mod friends
```

OUTPUT

```
Each friend gets 6
Left over: 3
```

**Try it!**

- [ ] Make a variable called `lives` set to 3. Take one away with `change lives by -1`.
- [ ] Work out how many days there are in 7 weeks.

# 3 Ask questions

Programs that listen to you

`ask` shows a question and a box to type the answer in. The word `into` tells me which variable to keep the answer in.

CODE

```
ask "What is your name?" into name
say "Hello, " + name + "!"
ask "How old are you?" into age
say "Next year you will be " + (age +
```

OUTPUT

```
What is your name? Mira
Hello, Mira!
How old are you? 9
Next year you will be 10
```

The green words in the output box are what the person typed.

**Sparky:** If you type a number, I keep it as a number, so I can do maths with it. That's how `age + 1` worked!

If you leave out `into`, I keep the answer in a variable called `answer`:

CODE

```
ask "What is your favourite colour?"
say answer + " is a lovely colour."
```

OUTPUT

```
What is your favourite colour? blue
blue is a lovely colour.
```

**Try it!**

- [ ] Ask for two numbers and say what they add up to.
- [ ] Make a "silly story" program that asks for an animal, a food and a place, then tells a story using them.

# 4 Make choices

if, else if and else

`if` asks a yes-or-no question. If the answer is **yes**, I do the instructions inside the block. If not, I skip them.

CODE

```
ask "How many points did you get?" into points
if points > 10:
  say "Amazing! You win a star!"
end
say "Thanks for playing."
```

OUTPUT

```
How many points did you get? 15
Amazing! You win a star!
Thanks for playing.
```

Add `else` for "otherwise", and `else if` to ask another question:

CODE

```
ask "How hot is it today?" into temp
if temp is bigger than 28:
  say "Wear a hat and drink water!"
else if temp is smaller than 12:
  say "Wear a jumper."
else:
  say "Perfect weather to play outside
end
```

OUTPUT

```
How hot is it today? 30
Wear a hat and drink water!
```

## Question words

| Write this | It means |
|---|---|
| `a is b` | Are they the same? Big and small letters count as the same, so "Yes" is "yes". |
| `a is not b` | Are they different? |
| `a is bigger than b` | Is a more than b? You can write `a > b` too. |
| `a is smaller than b` | Is a less than b? You can write `a < b` too. |
| `a ⩾ b, a ⩽ b` | at least, at most |
| `list contains thing` | Is the thing in the list (or in the words)? |
| `and, or, not` | Join questions together |

CODE

```
ask "Do you have a ticket?" into ticket
ask "How old are you?" into age
if ticket is "yes" and age ⩾ 10:
  say "You can ride the big rollercoaster!"
else:
  say "Let's try the carousel."
end
```

OUTPUT

```
Do you have a ticket? yes
How old are you? 11
You can ride the big rollercoaster!
```

**Try it!**

- [ ] Ask for a password. Say "Welcome!" if it is right, and "Go away!" if it is wrong.
- [ ] Ask for a number and say if it is even or odd. Hint: `number mod 2 is 0` means even.

# 5 Repeat with loops

Make Sparky do things again and again

Writing the same line 100 times is boring. A **loop** repeats a block for you.

CODE

```
repeat 3 times:
  say "Hip hip hooray!"
end
```

OUTPUT

```
Hip hip hooray!
Hip hip hooray!
Hip hip hooray!
```

## Repeat until

`repeat until` keeps going until the question becomes **yes**:

CODE

```
set count to 5
repeat until count is 0:
  say count
  change count by -1
end
say "Blast off!"
```

OUTPUT

```
5
4
3
2
1
Blast off!
```

`repeat while` is the opposite. It keeps going while the question is **yes**:

CODE

```
set money to 1
repeat while money < 100:
  set money to money * 2
end
say "Doubling 1 again and again gives " + money
```

OUTPUT

```
Doubling 1 again and again gives 128
```

## Repeat forever, stop and skip

`repeat forever` never ends by itself. Press **Stop** to end it, or use `stop` inside the loop to jump out. Always put a `wait` inside a forever loop so I can take a breath.

CODE

```
set n to 0
repeat forever:
  change n by 1
  if n mod 2 is 0:
    skip   # jump to the next round
  end
  if n > 9:
    stop   # leave the loop
  end
  say n + " is odd"
end
```

OUTPUT

```
1 is odd
3 is odd
5 is odd
7 is odd
9 is odd
```

**Try it!**

- [ ] Count from 1 to 20.
- [ ] Say the 5 times table: 5, 10, 15 … up to 50.
- [ ] Ask "Are we there yet?" again and again until somebody types "yes".

# 6 Lists and random surprises

Keep lots of things together

A **list** keeps many things in order, inside square brackets `[ ]` with commas between them. Item 1 is the first one.

CODE

```
set bag to ["apple", "book", "torch"]
add "snack" to bag
remove "book" from bag
say bag
say "First thing: " + item 1 of bag
say "Things in my bag: " + length of b
if bag contains "torch":
  say "Good, I can see in the dark!"
end
```

OUTPUT

```
[apple, torch, snack]
First thing: apple
Things in my bag: 3
Good, I can see in the dark!
```

You can also swap one item: `set item 2 of bag to "map"`.

## For each

`for each` runs a block once for every item. The name after `each` holds the current item.

CODE

```
set planets to ["Mercury", "Venus", "Earth", "Mars"]
for each planet in planets:
  say "Hello, " + planet + "!"
end
```

OUTPUT

```
Hello, Mercury!
Hello, Venus!
Hello, Earth!
Hello, Mars!
```

## Random surprises

`random 1 to 6` picks a number like rolling a dice. `random item from` picks one thing from a list. You get a different answer each time!

CODE

```
say "Dice: " + random 1 to 6
set moves to ["rock", "paper", "scissors"]
say "Sparky plays " + random item from moves
```

OUTPUT

```
Dice: 1
Sparky plays rock
```

**Try it!**

- [ ] Make a list of your 5 favourite animals and say each one.
- [ ] Make a "magic 8 ball": ask a question, then pick a random answer from a list.

# 7 Draw with Sparky

I carry a pen wherever I go

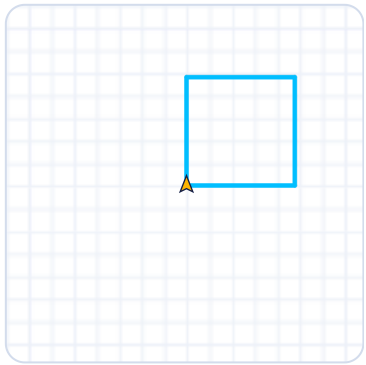
On my stage I start in the **middle**, facing **up**. When I walk, my pen draws a line. The middle is 0, 0, and the edges are 200 steps away.

| Instruction | What I do |
|---|---|
| `forward 100` | walk 100 steps forwards |
| `back 50` | walk backwards |
| `turn right 90` | turn on the spot. 90 is a quarter turn, 360 is a full turn. |
| `turn left 45` | turn the other way |
| `pen up / pen down` | lift the pen to move without drawing, then put it back |
| `color "red"` | change the pen colour |
| `pen size 8` | make the line thicker |
| `circle 40` | draw a circle around me |
| `go to 50, -100` | jump to a spot (across, then up) |
| `go home / clear` | go back to the middle / wipe the stage |

CODE

```
color "deepskyblue"
pen size 5
repeat 4 times:
  forward 120
  turn right 90
end
```

SPARKY'S STAGE

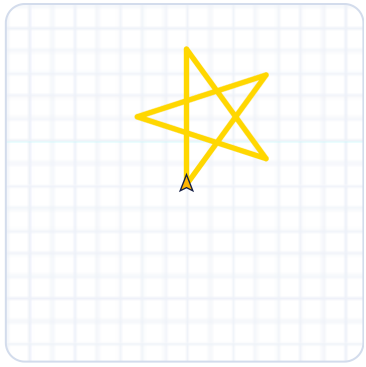


**Sparky:** **The shape trick:** to draw a shape with N sides, turn `360 / N` each time. A triangle turns 120 and a hexagon turns 60. A star turns 144!

CODE

```
color "gold"
pen size 6
repeat 5 times:
  forward 150
  turn right 144
end
```

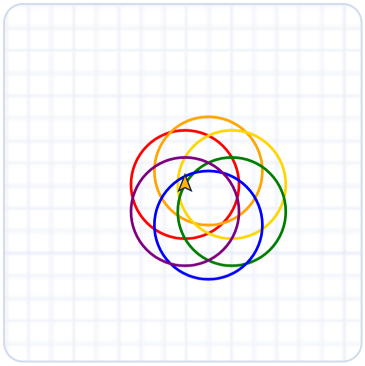
SPARKY'S STAGE



CODE

```
set colors to ["red", "orange", "gold", "green", "blue", "purple"]
pen size 3
for each c in colors:
  color c
  circle 60
  turn right 60
  pen up
  forward 30
  pen down
end
```

SPARKY'S STAGE



Colour names you can use include red, orange, gold, yellow, green, lime, teal, blue, deepskyblue, navy, purple, hotpink, brown, black and white. Grown-ups can also use codes like `"#33aa77"`.

**Try it!**

- [ ] Draw a triangle, then a hexagon.
- [ ] Draw a house: a square with a triangle roof.
- [ ] Make a rainbow spiral: `forward` a bit more each time round a loop.

# 8 Your own recipes

Teach Sparky new instructions

A **recipe** is a new instruction that you make up. Make it once with `define`, then use it as many times as you like. (Grown-up programmers call these **functions**.)

CODE

```
define cheer(name):
  say "Go " + name + "! You can do it!
end

cheer("Mira")
cheer("Arjun")
```

OUTPUT

```
Go Mira! You can do it!
Go Arjun! You can do it!
```

The words inside the brackets, like `name`, are **inputs**. A recipe can also send an answer back with `give back`:

CODE

```
define double(n):
  give back n * 2
end

define area(width, height):
  give back width * height
end

say double(21)
say "The rug covers " + area(3, 4) + " squares"
```

OUTPUT

```
42
The rug covers 12 squares
```

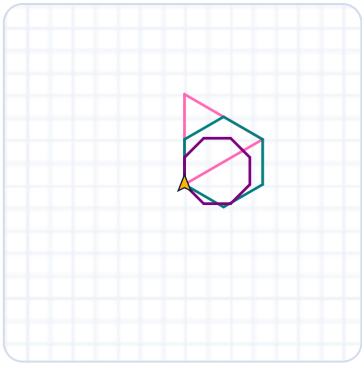
Recipes are great for drawing. This one can draw any shape:

CODE

```
define shape(sides, size):
  repeat sides times:
    forward size
    turn right 360 / sides
  end
end

color "hotpink"
shape(3, 100)
color "teal"
shape(6, 50)
color "purple"
shape(8, 30)
```

SPARKY'S STAGE



**Try it!**

- [ ] Make a recipe `add_up(a, b)` that gives back a + b.
- [ ] Make a recipe `flower(color_name)` that draws a flower in any colour.

# 9 Buttons and timing

Make things happen when you click

## Wait

`wait 1` makes me pause for 1 second. You can use halves too, like `wait 0.5`.

CODE

```
say "Ready..."
wait 0.2
say "Steady..."
wait 0.2
say "GO!"
```

OUTPUT

```
Ready...
Steady...
GO!
```

## Buttons

`when clicked "Jump":` makes a yellow button under my stage. Every time you click it, I run the block inside. Your program keeps listening for clicks until you press **Stop**.

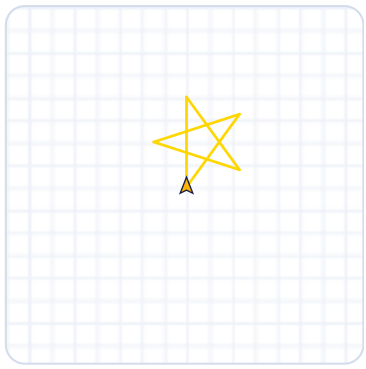
CODE

```
set score to 0
say "Click Jump!"

when clicked "Jump":
  change score by 1
  say "Boing! Score: " + score
end

when clicked "Star":
  color "gold"
  repeat 5 times:
    forward 100
    turn right 144
  end
end
```

SPARKY'S STAGE



OUTPUT

```
Click Jump!
(You click Jump, Jump, then Star.)
Boing! Score: 1
Boing! Score: 2
```

**Sparky:** Put `define` and `when` blocks at the left edge of your program, not inside other blocks.

**Try it!**

- [ ] Make a "Left" button and a "Right" button that turn me, and a "Go" button that moves me forward 20. Now you have a drawing game!
- [ ] Make a "Roll" button that rolls a dice.

# 10 Make a game

Characters, controls, sounds and scores

A game needs three things: **characters** to look at, **controls** to play with, and **rules** that decide when you win. In KidAI the characters are called **sprites**.

## Sprites

A sprite is an emoji or some words that can move around the stage. Give each sprite a name so you can move it later.

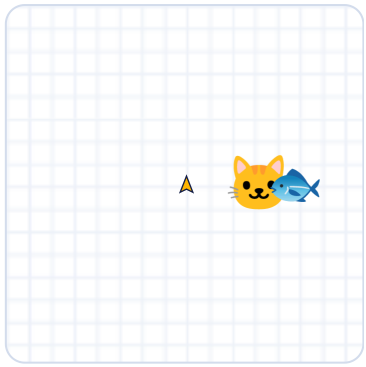
CODE

```
make sprite "cat" as "🐱" at -120, 0 size 60
make sprite "fish" as "🐟" at 120, 0
repeat until "cat" touches "fish":
  move sprite "cat" by 10, 0
end
say "Yum!"
```

OUTPUT

```
Yum!
```

SPARKY'S STAGE



| Write this | It means |
|---|---|
| `make sprite "cat" as "🐱"` | make a sprite (add `at x, y` and `size 60` if you like) |
| `move sprite "cat" by 10, 0` | move it a little: across, then up |
| `move sprite "cat" to 0, 0` | jump to a spot |
| `hide sprite / show sprite / delete sprite` | make it vanish, come back, or go for good |
| `"cat" touches "fish"` | yes if the two sprites are touching |
| `"cat" touches edge` | yes if the sprite reaches the side of the stage |
| `x of sprite "cat"` | where the sprite is (use `y of sprite` for up and down) |

**Sparky:** Make a sprite again with the same name to change how it looks: `make sprite "hero" as "😎"`. Words work as sprites too, which is handy for a score.

## Keyboard controls

There are two ways to use the keyboard. `when key ... pressed` runs a block once each time a key is pressed. `key ... is pressed` is a question you can ask again and again, for smooth movement. Arrow keys are called `"left"`, `"right"`, `"up"` and `"down"`.

CODE

```
make sprite "rocket" as "🚀" at 0, -150

when key "space" pressed:
  play sound "laser"
end

repeat forever:
  if key "left" is pressed:
    move sprite "rocket" by -8, 0
  end
  if key "right" is pressed:
    move sprite "rocket" by 8, 0
  end
  wait 0.03
end
```

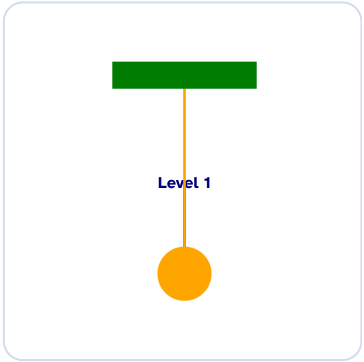
Click the stage first, then press the keys. On a tablet, KidAI shows buttons for the keys your game uses.

## Paint the stage

CODE

```
background "lightblue"
color "navy"
write "Level 1"
go to 0, -100
color "orange"
dot 30
go to 0, 120
color "green"
rectangle 160, 30
hide
```

SPARKY'S STAGE



## Sounds and music

Notes are the letters A to G. Add `#` for a sharp or a number for higher and lower notes, like `"C5"`. Sound effects are pop, coin, jump, boom, laser, win and lose.

CODE

```
for each n in ["C", "E", "G", "C5"]:
  play note n for 0.05
end
play sound "win"
```

OUTPUT

```
♪ C
♪ E
♪ G
♪ C5
♪ win!
```

## Timers

`every 1 second:` runs a block again and again, like a clock. `stop everything` ends the whole game.

CODE

```
set time_left to 10
every 1 second:
  change time_left by -1
  make sprite "clock" as time_left at 0, 0 size 120
  if time_left is 0:
    play sound "lose"
    stop everything
  end
end
```

## Put it together: catch the stars

CODE

```
background "midnightblue"
hide
make sprite "basket" as "🧺" at 0, -170 size 60
make sprite "star" as "⭐" at 0, 200 size 40
set score to 0
repeat forever:
  if key "left" is pressed:
    move sprite "basket" by -8, 0
  end
  if key "right" is pressed:
    move sprite "basket" by 8, 0
  end
  move sprite "star" by 0, -4
  if "star" touches "basket":
    change score by 1
    play sound "coin"
    move sprite "star" to random -180 to 180, 200
  end
  wait 0.03
end
```




Open "Game: catch the stars" in the Examples menu to play the full game with a score.

**Try it!**

- [ ] Make the star fall faster each time you catch it.
- [ ] Add a bomb sprite 💣 that ends the game if you catch it.
- [ ] Make a "Space" key that makes your basket jump.

# 11 Teach an AI

Computers that learn from examples

So far, you told me **exactly** what to do. **AI** (artificial intelligence) is different. You don't write the rules. You give the computer lots of **examples**, and it finds the pattern by itself. That is called **training**.

| AI word | What it does |
|---|---|
| `teach "label" with [...]` | gives the AI examples of one group (a **label**) |
| `guess "some words"` | asks the AI which label fits best |
| `how sure` | how confident the AI was, from 0 to 100 |

## Build a mood robot

CODE

```
teach "happy" with ["I love this", "yay so fun", "this is great", "I am smilin
teach "sad" with ["I feel bad", "this is awful", "I am crying", "so sad today"

ask "Tell me about your day" into sentence
set mood to guess sentence
say "I think you feel " + mood
say "I am " + how sure + "% sure."
```

OUTPUT

```
Tell me about your day today was so fun, I love my new bike
```

**AI guess: happy** · 67% sure · learned from 8 examples




```
I think you feel happy
I am 67% sure.
```

The pink card is the AI showing its work. It tells you how sure it is about **each** label, and how many examples it learned from.

**Sparky:** My AI learns from the **words** in your examples. It sees "fun" and "love" in your sentence, and those words were in the happy examples. It doesn't truly understand feelings the way you do.

You always need **at least two labels**. An AI can't choose if there is only one choice!

## Let people teach it

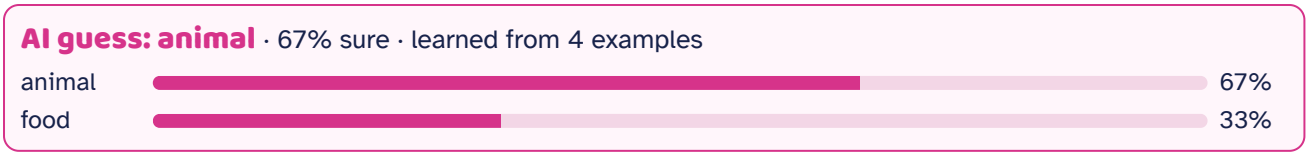
Real AI gets its examples from people. Your program can collect examples too:

CODE

```
repeat 4 times:
  ask "Type an animal or a food" into thing
  ask "Is it animal or food?" into kind
  teach kind with thing
end
ask "Give me something to guess" into word
say "That is a " + guess word
```

OUTPUT

```
Type an animal or a food cat
Is it animal or food? animal
Type an animal or a food pizza
Is it animal or food? food
Type an animal or a food lion
Is it animal or food? animal
Type an animal or a food rice
Is it animal or food? food
Give me something to guess a big lion
```




```
That is a animal
```

# 12 Is the AI right?

Testing AI, and catching its mistakes

Real AI scientists never just trust their AI. They **test** it with examples where they already know the right answer, and count how many it gets right. That number is called **accuracy**.

CODE

```
teach "happy" with ["I love this", "yay so fun", "this is great"]
teach "sad" with ["I feel bad", "this is awful", "I am crying"]

set tests to ["I love my dog", "I feel awful", "so much fun"]
set answers to ["happy", "sad", "happy"]
set correct to 0
set n to 1
for each sentence in tests:
  if guess sentence is item n of answers:
    change correct by 1
  end
  change n by 1
end
say "The AI got " + correct + " out of " + length of tests
```

OUTPUT

**AI guess: happy** · 57% sure · learned from 6 examples

| | |
|---|---|
| happy | 57% |
| sad | 43% |

**AI guess: sad** · 86% sure · learned from 6 examples

| | |
|---|---|
| sad | 86% |
| happy | 14% |

**AI guess: happy** · 80% sure · learned from 6 examples

| | |
|---|---|
| happy | 80% |
| sad | 20% |

```
The AI got 3 out of 3
```

## Catch the AI making a mistake

CODE

```
teach "happy" with ["I love this", "yay so fun", "this is great"]
teach "sad" with ["I feel bad", "this is awful", "I am crying"]
say guess "I am not happy"
say guess "my homework is great"
```

OUTPUT

**AI guess: sad** · 75% sure · learned from 6 examples

| | |
|---|---|
| sad | 75% |
| happy | 25% |

sad

**AI guess: happy** · 67% sure · learned from 6 examples

| | |
|---|---|
| happy | 67% |
| sad | 33% |

happy

The first answer, **sad**, is right, but for the wrong reason! The AI matched the words "I am", which were in a sad example ("I am crying"). It has no idea what "not" means. The second answer is "happy" only because of the word "great". Is homework really happy?

## Unfair examples make an unfair AI

If one label gets many more examples than the other, the AI starts to **lean** towards it. This is called **bias**. Watch what happens with words the AI has never seen:

CODE

```
teach "cat" with ["meow", "purr", "fluffy", "nap", "tail"]
teach "dog" with ["woof", "bark"]
say guess "hello there"
```

OUTPUT

**AI guess: cat** · 71% sure · learned from 7 examples

| | |
|---|---|
| cat | 71% |
| dog | 29% |

I have never seen these words, so this is just a guess.

cat

"hello there" is not about cats at all, but the AI says **cat** because it saw more cat examples.

**Sparky's AI rules**

1. AI can be wrong, so always check its answers.
2. Give every label lots of different examples, and about the same number each.
3. A high "how sure" doesn't mean it's right.
4. Never teach an AI secrets, like passwords or your home address.

**Try it!**

- [ ] Teach the AI to tell "sport" from "music". Test it with 5 sentences. What's its accuracy?
- [ ] Find a sentence that tricks your AI. Then add examples so it gets it right.
- [ ] Make a weather AI: "sunny", "rainy" and "snowy".

# 13 Pictures, numbers and big AI

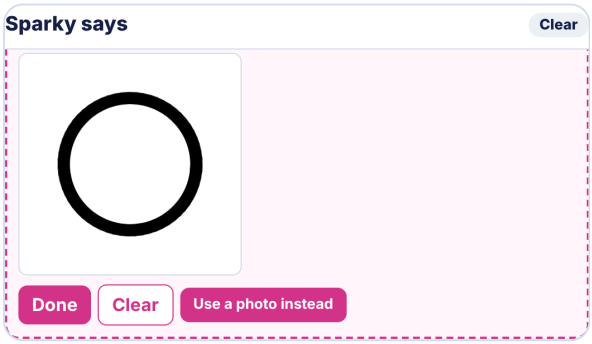
More ways for computers to learn

## An AI that learns from pictures

`ask picture "Draw a cat" into cat1` opens a drawing pad. You can draw with your mouse or finger, or use a photo. Then you teach the AI with pictures, just like with words.

CODE

```
ask picture "Draw a circle" into c1
ask picture "Draw a square" into s1
teach "circle" with [c1]
teach "square" with [s1]
ask picture "Draw one more!" into mystery
say guess mystery
```




The drawing pad appears where Sparky talks.

Sparky can also make training pictures by itself. `stage picture` takes a picture of whatever Sparky has drawn:

CODE

```
define circle_picture(size):
  clear
  circle size
  give back stage picture
end
define line_picture(size):
  clear
  forward size
  give back stage picture
end
teach "circle" with [circle_picture(40), circle_picture(90)]
teach "line" with [line_picture(60), line_picture(150)]
test "circle" with [circle_picture(65), circle_picture(120)]
test "line" with [line_picture(100)]
```

OUTPUT

```
AI test: 2 / 2 (100%)
AI test: 0 / 1 (0%)
Wrong: a picture → circle
```

**Sparky:** A picture AI shrinks every picture to 16 × 16 squares and looks for the pictures that are most alike. It doesn't know what a cat is, only which pictures look similar.

## An AI that learns from numbers

Give `learn` pairs of numbers: an **input** and an **answer**. The AI finds the pattern, and `predict` uses it for a new input.

CODE

```
# [days of watering, height in cm]
learn "plant" from [[1, 2], [2, 4.1], [3, 5.9], [4, 8.2]]
say predict "plant" for 10
say how sure + "% sure the pattern fits"
```

OUTPUT

**AI prediction: 20.35** · 100% sure · pattern: answer = 2.04 × input + -0.05

```
20.35
100% sure the pattern fits
```

The AI draws a straight line through your examples. Real plants stop growing one day, so a prediction for 1,000 days would be silly. **Always ask: does this answer make sense?**

## Save what the AI learned

CODE

```
teach "yes" with ["yep", "sure"]
teach "no" with ["nope", "never"]
save ai as "my answers"
forget ai
load ai "my answers"
say guess "sure thing"
```

OUTPUT

```
(Saved the AI as "my answers")
(Loaded the AI "my answers")
```

**AI guess: yes** · 67% sure · learned from 4 examples

yes 67%

no 33%

```
yes
```

## Chat with a big AI

`chat` sends a question to a big AI on the internet (Claude) and gives back its answer:

CODE

```
ask "What do you want to know?" into question
say chat question
```

A big AI was trained on huge amounts of writing, so it can answer almost anything. Your tiny AI only knows your examples, but you can see exactly why it guesses what it does. Compare

them!

**Big AI rules**

1. chat only works when a grown-up has switched it on.
2. Your question leaves your device, so never type your name, school, address or passwords.
3. Big AIs can sound very sure and still be wrong. Check important answers in a book or with a grown-up.

**Try it!**

- [ ] Teach a picture AI to tell happy faces from sad faces.
- [ ] Use `learn` to predict how many sweets you will have after saving some every day.

# 14 Records, words and saving

Tools for bigger programs

## Records

A **record** keeps facts about one thing together. Each fact has a name, and you read it with a dot.

CODE

```
set pet to {name: "Tom", kind: "cat", age: 3}
say pet.name + " is a " + pet.kind
set pet.age to pet.age + 1
set pet.toy to "ball"
say pet
say keys of pet
```

OUTPUT

```
Tom is a cat
{name: Tom, kind: cat, age: 4, toy: ball}
[name, kind, age, toy]
```

## Word tools

CODE

```
set colours to split "red,green,blue" by ","
say colours
say join colours with " and "
say replace "like" with "love" in "I like cats"
say "Hello" starts with "he"
say "photo.png" ends with ".jpg"
```

OUTPUT

```
[red, green, blue]
red and green and blue
I love cats
yes
no
```

## Saving for next time

Variables are forgotten when a program stops. `save` keeps something even after you close the page, and `load` brings it back.

CODE

```
load "best" into best
if best is nothing:
  set best to 0
end
set score to 42
if score > best:
  save score as "best"
  say "New high score!"
end
```

OUTPUT

```
New high score!
```

## Borrow recipes

When your recipes get useful, put them in their own project, say "my helpers". Then any program can borrow them with `use "my helpers"` on its first line.

**Try it!**

- [ ] Make a record for your favourite book, with its title, writer and number of pages.
- [ ] Ask for a sentence and say how many words it has. Hint: `length of split sentence by " "`.

# 15 Watch your code run

Step, memory, blocks and grown-up code

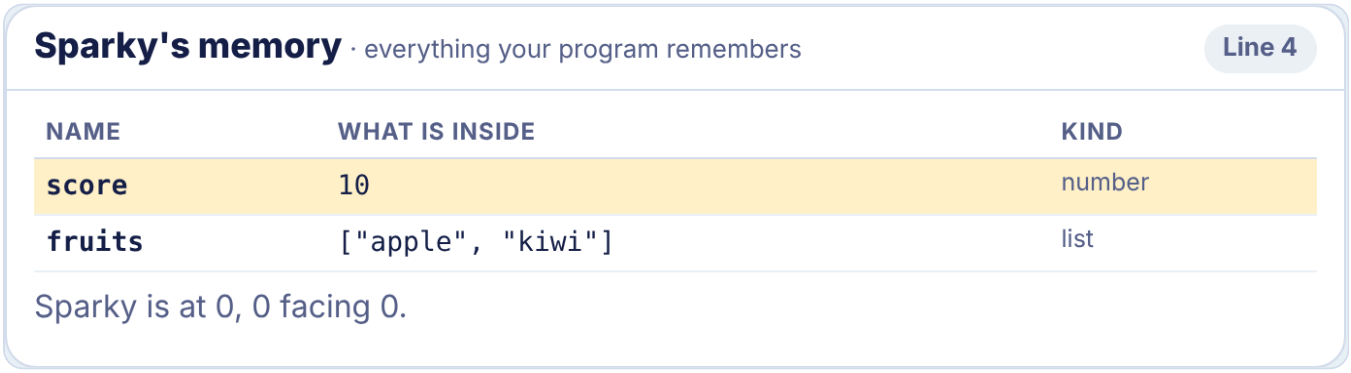
## One line at a time

Press the yellow **Step** button instead of Run. Sparky does one line, then waits. The line that runs next glows yellow. Keep pressing Step to go on, or press Run to finish at full speed. The speed menu also has **Slow** and **Very slow**.



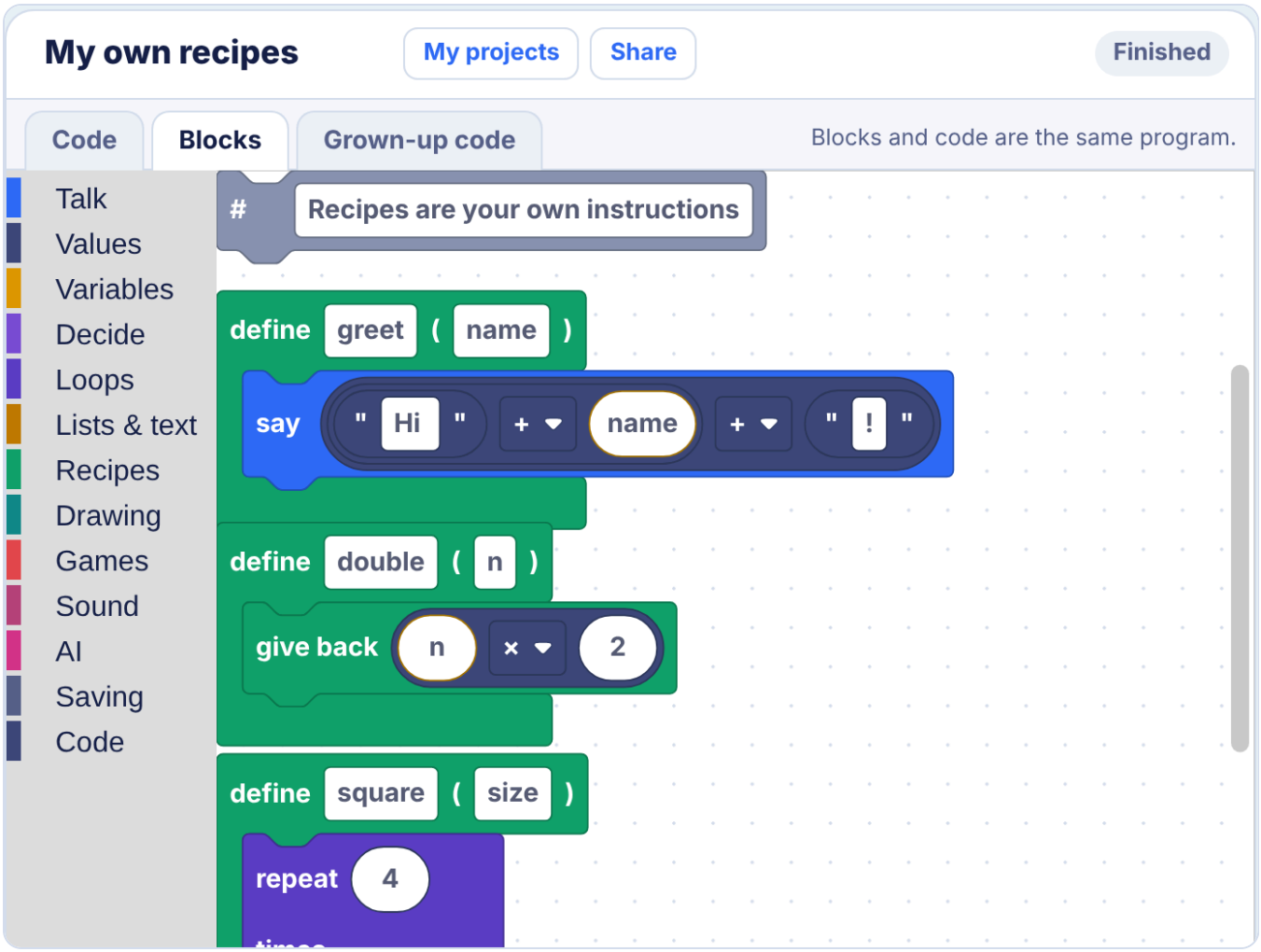

## Sparky's memory

Under your code, **Sparky's memory** shows every variable and what is inside it. Values that just changed flash yellow. When a recipe runs, its inputs appear in their own box.




## Blocks

The **Blocks** tab shows your program as coloured blocks you can drag around. Blocks and code are the same program: change one and the other changes too. The blocks light up while they run.




## Grown-up code

The **Grown-up code** tab shows how the same program looks in a bigger language. It is for reading: spot the differences!

**My own recipes** My projects Share Finished

Code | Blocks | **Grown-up code** | For reading: this is how bigger languages write the same ideas.

```
# Your program in grown-up KidAI Core 1.0 style.
# It shows the same ideas the way a bigger language writes them.
# Some library names here are examples, so this version is for reading, not runn

use draw.turtle

# Recipes are your own instructions

function greet(name: Any) -> Nothing:
    show "Hi " + to_text(name) + "!"
end

function double(n: Any) -> Any:
    return n * 2
end
```

**Sparky:** In **Settings** you can switch the playground to हिन्दी (Hindi), make the text bigger, and turn on typing keys for phones and tablets.

# 16 Fix bugs like a pro

Every programmer makes mistakes

A mistake in code is called a **bug**. Finding and fixing it is called **debugging**. Even expert programmers get bugs every day! When something is wrong, I say **Oops**, tell you the line number, and give you a tip.

CODE

```
set score to 5
say scroe
```

SPARKY SAYS

**Oops! Line 2**

I don't know anything called "scroe" yet.

**Tip:** Did you mean "score"?

CODE

```
sya "Hello"
```

SPARKY SAYS

**Oops! Line 1**

I don't know the instruction "sya".

**Tip:** Did you mean "say"?

CODE

```
repeat 3 times:
  say "hi"
```

SPARKY SAYS

**Oops! Line 1**

This "repeat" needs an "end" to finish it.

**Tip:** Add a line that says end after the instructions inside it.

CODE

```
say "Hello
```

SPARKY SAYS

**Oops! Line 1**

This text is missing its closing quote mark.

**Tip:** Put a " at the end of the words, like: say "Hello"

CODE

```
set pets to ["cat"]
say item 2 of pets
```

SPARKY SAYS

**Oops! Line 2**

There is no item 2. The list only has 1 item.

**Tip:** Use item 1.

CODE

```
say 10 / 0
```

SPARKY SAYS

**Oops! Line 1**

You can't divide by zero. Not even a computer can!

**Tip:** Use a number other than 0.

CODE

```
set n to 1
repeat until n is 0:
  change n by 1
end
```

SPARKY SAYS

**Oops! Line 2**

This program used too many steps, so I stopped it safely.

**Tip:** Look for a loop that never ends. If it should run forever, add "wait 0.1" inside it.

That last loop never ends, because `n` only gets bigger. I stop it safely so your computer doesn't freeze.

## Bug-hunting checklist

- [ ] Read the Oops message and look at the red line.
- [ ] Check spelling, especially variable names.
- [ ] Do all your words have a quote mark at both ends?
- [ ] Does every block (repeat, if, for, define, when) have its own `end`?
- [ ] Did you `set` a variable before using it?
- [ ] Still stuck? Read your code out loud, line by line. This trick really works!

# 17 Share and take on challenges

Code with friends and your class

## Your projects

Click the name above your code to rename your project. **My projects** lists everything you have made on this device. You can start a new one, make a copy, or open and save `.kid` files.

## Share and remix

Press **Share** to get a **share code** that starts with `KID1`. Send it to a friend. They paste it into the same box and press **Open as a remix** to get their own copy. Their changes never change yours.

## Class challenges

Your teacher can turn a program into a **challenge**. When you open a challenge code, the task appears above your code. Press **Check my work** to see which tests pass, and **Hand it in** to make a code for your teacher.




**Try it!**

- [ ] Share your favourite drawing program with a friend and remix theirs.
- [ ] Make a challenge for a friend: write a program, then use **Make a challenge from this program** in My projects.

# 18 Big projects

Put it all together

## Project 1: Guess my number

CODE

```
set secret to random 1 to 20
set tries to 0
set guess_number to 0
say "I'm thinking of a number from 1 to 20."

repeat until guess_number is secret:
  ask "Your guess?" into guess_number
  change tries by 1
  if guess_number is bigger than secret:
    say "Too big!"
  else if guess_number is smaller than secret:
    say "Too small!"
  end
end
say "You got it in " + tries + " tries!"
```

OUTPUT

```
I'm thinking of a number from 1 to 20.
Your guess? 10
Too small!
Your guess? 15
You got it in 2 tries!
```

## Project 2: Quiz master

CODE

```
set questions to ["Capital of France?", "How many legs does a spider have?", "B
set answers to ["Paris", "8", "Jupiter"]
set score to 0
set n to 1
for each q in questions:
  ask q into reply
  if reply is item n of answers:
    say "Correct!"
    change score by 1
  else:
    say "Oops, it was " + item n of answers
  end
  change n by 1
end
say "You scored " + score + " out of " + length of questions
```

OUTPUT

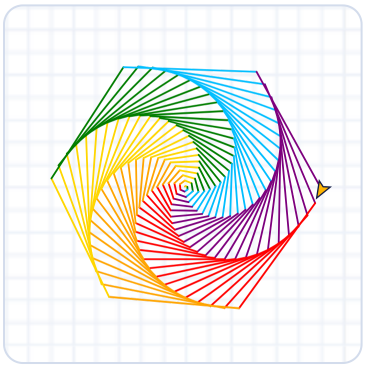
```
Capital of France? Paris
Correct!
How many legs does a spider have? 8
Correct!
Biggest planet? Jupiter
Correct!
You scored 3 out of 3
```

## Project 3: Spiral art

CODE

```
set colors to ["red", "orange", "gold", "green", "deepskyblue", "purple"]
pen size 2
set i to 0
repeat 150 times:
  color item (i mod 6) + 1 of colors
  forward i
  turn right 59
  change i by 1
end
```

SPARKY'S STAGE



**Make it your own!**

- [ ] Guess my number: give the player only 5 tries.
- [ ] Quiz master: add your own questions about your favourite topic.
- [ ] Spiral art: change 59 to 89 or 121 and see what happens!

# Sparky's word book

A–Z

Every word I know, on one page

## Talking

```
say "Hi"
ask "Name?" into name
wait 1     # seconds
```

## Remembering

```
set score to 0
change score by 1
add 5 to score
```

## Maths

```
+  -  *  /  mod  ( )
round(7.6)   sqrt(16)   abs(-3)
number("42")   text(42)
```

## Choosing

```
if a is b:
else if a is bigger than b:
else:
end
is not   >   <   ⩾   ⩽
and   or   not   contains
```

## Looping

```
repeat 3 times:
repeat until x is 5:
repeat while x < 5:
repeat forever:
for each pet in pets:
stop    skip    end
```

## Lists

```
set pets to ["cat", "dog"]
add "fish" to pets
remove "cat" from pets
item 1 of pets
set item 1 of pets to "owl"
length of pets
random 1 to 6
random item from pets
```

## Drawing

```
forward 100      back 50
turn right 90    turn left 90
pen up           pen down
pen size 5       color "red"
circle 30        clear
go to 50, 100    go home
hide             unhide
```

## Recipes and buttons

```
define greet(name):
  give back "Hi " + name
end
when clicked "Jump":
end
```

## AI

```
teach "happy" with ["yay", "fun"]
guess "so much fun"     how sure
test "happy" with ["fun day"]
how accurate
ask picture "Draw a cat" into cat
stage picture
learn "plant" from [[1, 2], [2, 4]]
predict "plant" for 5
chat "Why is the sky blue?"
save ai as "moods"    load ai "moods"
forget ai
```

## Games

```
make sprite "cat" as "🐱" at 0, 0 size 50
move sprite "cat" by 10, 0
move sprite "cat" to 0, 0
hide sprite "cat"   show sprite "cat"
"cat" touches "fish"
"cat" touches edge
x of sprite "cat"   y of sprite "cat"
when key "space" pressed:
key "left" is pressed
every 1 second:
stop everything
```

## Sound and stage

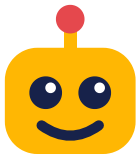
```
play note "C" for 0.5
play sound "coin"     beep
background "skyblue"
write "Hi!"   dot 20
rectangle 80, 40   turn to 90
x position   y position   direction
```

## Records, words, saving

```
set pet to {name: "Tom"}
pet.name    set pet.age to 3
keys of pet
split "a,b" by ","
join pets with ", "
replace "a" with "o" in word
word starts with "he"
save score as "best"
load "best" into best
use "my helpers"
```

## Words and values

```
"words in quotes"   42   3.5
yes   no   nothing
uppercase("hi")   lowercase("HI")
# a note for humans
```



# KidAI Coder Certificate

This is to say that

_____

has learned to code with Sparky, and can now:

- ○ make Sparky talk and do maths
- ○ remember things with variables
- ○ ask questions and use the answers
- ○ make choices with if
- ○ repeat things with loops
- ○ use lists and random surprises
- ○ draw shapes and art
- ○ make my own recipes
- ○ make buttons
- ○ teach and test an AI
- ○ spot AI mistakes and bias
- ○ make a game with sprites and keys
- ○ teach an AI with pictures and numbers
- ○ use records and save data
- ○ watch my code run step by step
- ○ fix bugs like a pro

_____ _____

Date Signed by Sparky's helper

# Meet the developer

KidAI Easy is developed by Sourabh Somani




## Sourabh Somani

KidAI Easy and this book were developed by Sourabh Somani. He is a software developer and tech lead from India, and he studied computer science. He builds websites and apps, speaks at technology events, and writes articles that help other people learn to code.

Microsoft has named him a **Microsoft MVP** six times, and the C# Corner community has named him a **C# Corner MVP** eight times. MVP means "Most Valuable Professional". These awards thank people who share what they know and help others in the coding world.

**Sparky:** Sourabh developed me so that kids everywhere can make games, art and their very own AI, using words that read like English. Who knows? One day you might invent your own programming language too!

## Find Sourabh online

Website
**sourabhsomani.com**

GitHub
**github.com/sourabhsomani**

KidAI Easy code
**github.com/sourabhsomani/kidai-easy**

Blog
**blogs.sourabhsomani.com**

LinkedIn
**linkedin.com/in/sourabh-somani-28969234**

X (Twitter)
**@sourabh_somani**

**Stay safe online**
Always ask a grown-up before you visit a website or social media page.

Found a bug in KidAI Easy, or have an idea to make it better? Ask a grown-up to help you tell Sourabh on GitHub.

# For grown-ups

Parents, teachers and helpers

**KidAI Easy** is the beginner level of the KidAI language. It's for children aged about 8 and up, and it keeps the KidAI foundations: blocks open with `:` and close with `end`, `#` comments, `.kid` files, and offline-first design. It removes types, async code and symbol-heavy syntax so a child can get a first program working in minutes.

## Safety by design

- Programs can't reach files, the camera or the microphone, and can't run any other code. The only way out to the internet is `chat`, which is off unless a grown-up turns it on: with the chat gateway (see the README), by setting `ANTHROPIC_API_KEY` before running the `kidai` command, or by opening the playground through a Claude account.
- Drawings and photos for the picture AI stay on the device. Projects and saved data live in the browser; share codes contain only the program.
- The tiny AI is a small word-counting classifier (naive Bayes). It learns only from the examples in the child's own program, runs on the device, and forgets everything when the program stops unless the program uses `save ai`. Every guess shows its confidence for each label and how many examples it used.
- A program can take 1,000,000 steps between pauses, which catches endless loops. Recipes can go 400 calls deep, and a wait lasts at most 60 seconds. The Stop button always works.

## Teaching tips

- Work through one chapter per session. Let children type every example themselves rather than copy-pasting it, because typing mistakes teach debugging.
- Chapters 11 to 13 are about AI literacy: learning from data, testing, accuracy, confidence, bias, and the difference between a tiny AI you trained and a big cloud AI. Ask children to explain *why* the AI made each guess.
- Kind error messages are part of the lesson. Encourage children to read the "Oops" and the tip before asking for help.
- The `kidai` command (install with `npm install -g kidai-easy`) runs programs in a terminal. Programs that draw, play sound, use keys or buttons, or ask for pictures open a stage window in the web browser. It also adds `kidai check`, `kidai fmt`, `kidai svg` (save drawings) and `kidai --lang hi`.
- In VS Code, search the Extensions panel for **KidAI Easy - Coding for Kids with Sparky**. It adds colours, ready-made snippets and a Sparky icon for `.kid` files. The code, the online playground and this guide are at **github.com/sourabhsomani/kidai-easy**.
- Class challenges need no accounts: your program is the answer key, students check and hand in with codes, and **Check students' work** marks a whole class at once.
- Blocks mode downloads the Blockly library the first time it is opened, so it needs the internet once.

## Language rules at a glance

- Keywords and names ignore capital letters. "Curly" quotes from phones and tablets work too.
- Lists start at item 1. `+` joins text and numbers. Typed answers that look like numbers become numbers.
- `is` compares text without caring about capital letters or extra spaces. `if` needs a real yes/no value.
- Inside a recipe, `set` changes a variable from the main program if one exists with that name. Otherwise it makes a new variable inside the recipe.